

UNIVERSITY OF MINES AND TECHNOLOGY
FACULTY OF COMPUTING AND MATHEMATICAL SCIENCES
DEPARTMENT OF CYBERSECURITY AND INFORMATION SYSTEMS



REPORT ENTITLED
BUILDING A DISTRIDUTED IDS FOR MULTI SEGMENT NETWORK MONITORING

BY
NAME: BOTUO THERESAH

INDEX NUMBER: FCM.41.018.109.23

COURSE: NETWORK SECURITY, AUDITING AND MONITORING (CY376)
BLUE TEAM
CLASS: CY III

SEMESTER: TWO

COURSE INSTRUCTOR
FREDERICK EDEM BRONI (MR)

TARKWA, GHANA

AUGUST 2026

Abstract

Modern enterprise networks are commonly divided into multiple segments — such as a demilitarized zone (DMZ), an internal user network, and a management or security-operations segment — to contain the impact of a compromise in any single zone. However, segmentation alone does not provide visibility into malicious activity occurring within or between these zones. This project addresses that gap by designing, implementing, and evaluating a distributed intrusion detection system (IDS) for a three-segment network. The system deploys a dedicated Suricata sensor within each segment to inspect local traffic in real time, with all sensors forwarding alerts to a centralized Wazuh manager for correlation, storage, and visualization. A pfSense firewall/router provides inter-segment routing and access control, while a Kali Linux host is used to generate representative attack traffic (port scans, vulnerability scans, and flooding attempts) for evaluation. [Summarize your key results here once testing is complete, e.g., number and types of alerts generated, sensor performance, any detection gaps identified.] The findings demonstrate [state your overall conclusion once available] and inform recommendations for extending distributed monitoring in segmented network environments.

Table of Contents

Abstract.....	2
1. Introduction.....	4
1.1 Problem Statement.....	4
1.2 Project Objectives	4
1.3 Scope	5
2. Literature and Tooling Review.....	5
2.1 Intrusion Detection Approaches	5
2.2 Suricata.....	5
2.3 Wazuh.....	5
2.4 MITRE ATT&CK	6
2.5 CIS Benchmarks.....	6
2.6 Related Architectural Approach: pfSense.....	6
3. Methodology	6
3.1 Lab Environment	6
3.2 Network Topology	6
3.3 Tools and Justification	8
3.4 Procedure	8
3.5 Deviations and Issues Encountered	8
4. Implementation.....	9
4.1 pfSense Configuration	9
4.2 Suricata Deployment	11
4.3 Connectivity Troubleshooting (Evidence)	14
8. References	16

1. Introduction

Network segmentation is a foundational control in modern security architecture: by dividing a network into zones with distinct trust levels — for example, a public-facing DMZ, an internal user LAN, and a management or security-operations segment — an organization limits how far an attacker can move after gaining an initial foothold. Segmentation, however, is a containment control, not a detection control. Without monitoring inside each segment and a way to correlate what is seen across segments, malicious activity can persist undetected for long periods, and analysts lose the ability to reconstruct how an incident unfolded across the network.

This project takes the Blue Team perspective on that problem. Rather than focusing on offensive techniques, the objective is to design and build a monitoring capability that gives a defender visibility into activity across every segment of a small, representative multi-segment network, and to centralize that visibility so it can be acted on from a single point rather than three disconnected logs.

1.1 Problem Statement

A segmented network without per-segment monitoring creates blind spots: traffic that never crosses the router/firewall boundary (for example, lateral movement between two hosts on the same internal segment) is invisible to a single, centrally placed IDS. A genuinely distributed IDS is needed, one sensor per segment, so that intra-segment as well as inter-segment traffic can be inspected.

1.2 Project Objectives

- Design a segmented network topology representative of a small organization (DMZ, internal LAN, management/SOC segment).
- Deploy a dedicated Suricata IDS sensor within each segment to provide local, real-time traffic inspection.
- Centralize alerting and correlation using a Wazuh manager, so that all sensors report to a single dashboard.
- Generate representative attack traffic (reconnaissance, vulnerability scanning, and flooding) to validate detection coverage.

- Evaluate the resulting system against recognized frameworks (MITRE ATT&CK, CIS Benchmarks) and produce recommendations for improvement.

1.3 Scope

The project is implemented entirely within an isolated virtual lab (VMware Workstation) using simulated hosts and traffic. No production systems, real user data, or unauthorized third-party targets are involved at any stage, consistent with the course's academic integrity requirements for lab-based work.

2. Literature and Tooling Review

2.1 Intrusion Detection Approaches

IDS technology is generally split into signature-based and anomaly-based detection. Signature-based systems match traffic against a database of known attack patterns and are effective against well-documented threats but blind to novel ones; anomaly-based systems instead model normal behavior and flag deviations, catching unknown attacks at the cost of a higher false-positive rate. Most production deployments, including the one built for this project, combine both approaches: a signature engine for known threats, supplemented by manager-side correlation rules that can flag unusual patterns across multiple sensors.

2.2 Suricata

Suricata is an open-source, multi-threaded network IDS/IPS engine capable of signature-based detection, protocol identification, and file extraction from live traffic. Its native structured logging format (EVE JSON) makes it straightforward to forward alerts into an external log management and correlation platform, which is the property this project relies on to connect three independent sensors to one central manager. Suricata's multi-threaded design also allows it to keep pace with higher traffic volumes than older single-threaded engines, which matters when a sensor is expected to inspect all traffic within a segment rather than a single link.

2.3 Wazuh

Wazuh is an open-source security monitoring platform that combines a central manager, an indexer for log storage/search, and a dashboard for visualization, alongside lightweight agents installed on monitored hosts. In this project, Wazuh serves as the correlation layer: each Suricata sensor's agent forwards the EVE JSON log to the manager, where alerts from all three segments become visible and searchable from a single dashboard. This centralization is the core

architectural idea being tested — that a distributed set of sensors is only useful to a defender if their output is unified rather than left as three separate logs.

2.4 MITRE ATT&CK

The MITRE ATT&CK framework catalogs adversary tactics and techniques observed in real intrusions and is widely used to structure both offensive testing and defensive coverage assessments. This project maps the attack traffic generated during testing (for example, network scanning and discovery activity) to relevant ATT&CK techniques, giving the results and findings section a standard vocabulary rather than ad-hoc descriptions.

2.5 CIS Benchmarks

The CIS Benchmarks provide vendor-specific, consensus-based hardening guidance for operating systems, network devices, and applications. Where relevant, this project's implementation and analysis sections reference applicable CIS guidance for the router/firewall and host configurations used, to ground configuration choices in a recognized standard rather than personal preference.

2.6 Related Architectural Approach: pfSense

pfSense is an open-source firewall/router distribution built on FreeBSD, offering stateful firewalling, routing between multiple interfaces, and DHCP/DNS services through a web-based interface. It is used here purely as the segmentation and routing backbone connecting the three lab segments; the security monitoring itself is deliberately kept separate, on the dedicated Suricata sensors, so that the router's role stays limited to traffic forwarding and access control rather than detection.

3. Methodology

3.1 Lab Environment

The lab was built entirely in VMware Workstation on a single host machine. All addressing, traffic, and attack activity described in this report is confined to this isolated virtual environment.

3.2 Network Topology

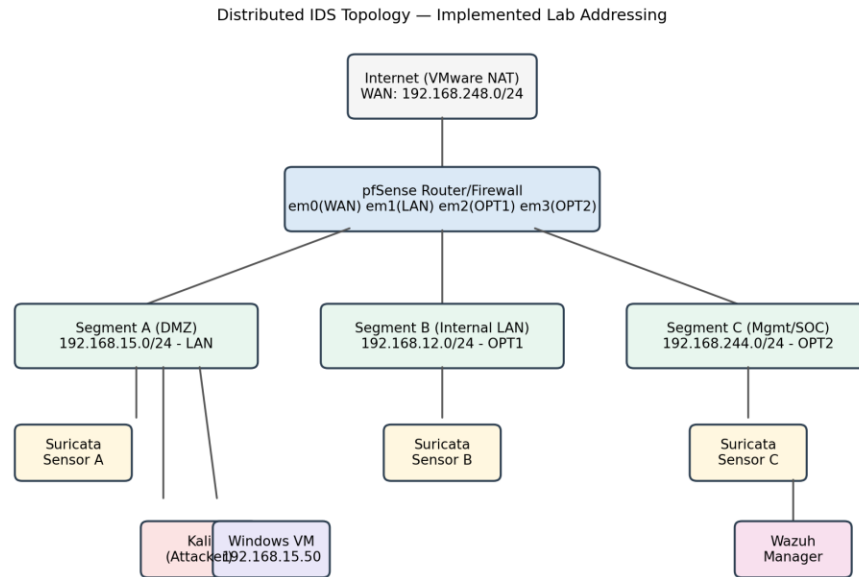


Figure 1: Distributed IDS network topology, showing the addressing actually implemented in the lab

The topology consists of a pfSense router/firewall connecting three isolated segments: Segment A (DMZ, 192.168.15.0/24), Segment B (Internal LAN, 192.168.12.0/24), and Segment C (Management/SOC, 192.168.244.0/24). These subnets were auto-assigned by VMware Workstation when the corresponding host-only virtual networks (VMnet2, VMnet3, VMnet4) were created, and were adopted as-is rather than forced to arbitrary values, since the specific subnet numbers are not architecturally significant. Each segment hosts a dedicated Suricata sensor. The Wazuh manager resides on Segment C, reflecting its role as the security-operations zone. A Kali Linux host on Segment A generates attack traffic representing an external or DMZ-originated threat.

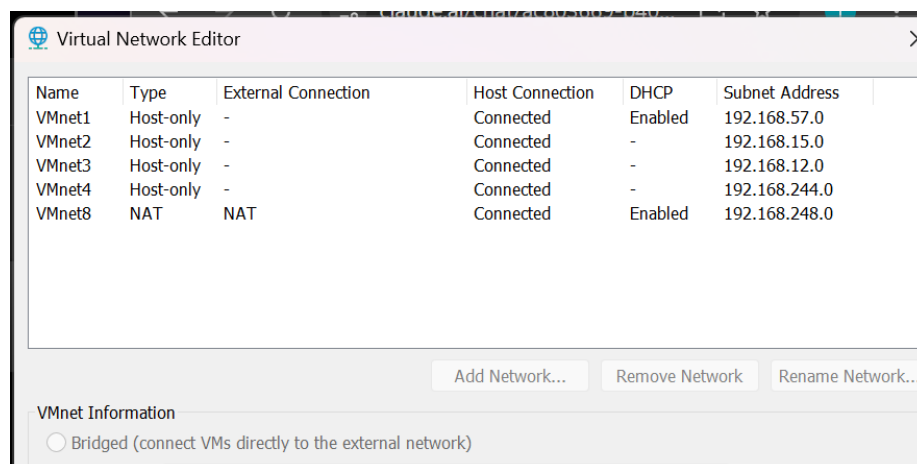


Figure 2: VMware Virtual Network Editor showing the three host-only segments (VMnet2, VMnet3, VMnet4) with DHCP disabled, since pfSense provides DHCP for each segment

3.3 Tools and Justification

Tool	Role in Lab	Justification
pfSense	Router/firewall connecting the three segments	Free, GUI-driven, widely documented; keeps routing and monitoring cleanly separated
Suricata	Per-segment IDS sensor	Multi-threaded, native JSON (EVE) logging integrates directly with Wazuh
Wazuh	Central alert correlation and dashboard	Combines manager, indexer, and dashboard; agent model matches distributed sensor design
Kali Linux	Attack traffic generation	Standard, well-documented toolset (nmap, hping3) for reproducible test cases

3.4 Procedure

The lab was built in the following sequence:

- Configured VMware virtual switches for the three isolated segments and the pfSense NAT/WAN link.
- Installed and configured pfSense, assigning WAN and three LAN interfaces with the addressing scheme above, and enabling DHCP per segment.
- Deployed one Ubuntu Server VM per segment and installed Suricata, setting HOME_NET to the local subnet and enabling EVE JSON logging.
- Deployed the Wazuh manager (indexer, server, dashboard) and registered a Wazuh agent on each Suricata sensor, configured to forward the eve.json log.
- Deployed a Kali Linux VM and generated reconnaissance and flooding traffic against hosts in each segment.
- Captured alerts from Suricata's local logs and the centralized Wazuh dashboard, and mapped observed activity to MITRE ATT&CK techniques.

3.5 Deviations and Issues Encountered

Two notable issues arose during the build and are documented here rather than omitted, since diagnosing them is itself part of the methodology:

- A VMware e1000 virtual NIC / FreeBSD checksum-offload interaction caused intermittent, silent ICMP failures between pfSense and directly connected hosts, despite correct IP addressing and successful ARP resolution. This was resolved by disabling

hardware checksum offload, TCP segmentation offload, and large receive offload under System > Advanced > Networking in pfSense, followed by a full reboot of the affected VMs.

- DNS resolution initially failed on the Segment A sensor (affecting both suricata-update's rule download and general internet reachability), traced to the same underlying offload/driver inconsistency; resolving it required disabling NIC offload features (ethtool) on the Ubuntu host and rebooting rather than restarting networking services alone.

4. Implementation

This section documents the concrete configuration of each component, with evidence captured directly from the lab as it was built.

4.1 pfSense Configuration

pfSense CE 2.8.1-RELEASE was installed as a VM with four network adapters: one on VMware NAT (WAN) and three custom adapters mapped to VMnet2/3/4 (Segments A, B, and C). During first boot, the four detected interfaces (em0-em3) were assigned to WAN, LAN, OPT1, and OPT2 respectively.

```

Do you want to proceed [y/n]? y

Writing configuration...done.
One moment while the settings are reloading... done!
VMware Virtual Machine - Netgate Device ID: d8b4400c5bd652fc684f

*** Welcome to pfSense 2.8.1-RELEASE (amd64) on pfSense ***

WAN (wan)   -> em0 -> v4/DHCP4: 192.168.248.151/24
LAN (lan)   -> em1 ->
OPT1 (opt1) -> em2 ->
OPT2 (opt2) -> em3 ->

0) Logout / Disconnect SSH          9) pfTop
1) Assign Interfaces                10) Filter Logs
2) Set interface(s) IP address      11) Restart GUI
3) Reset admin account and password 12) PHP shell + pfSense tools
4) Reset to factory defaults        13) Update from console
5) Reboot system                    14) Enable Secure Shell (sshd)
6) Halt system                      15) Restore recent configuration
7) Ping host                        16) Restart PHP-FPM
8) Shell

Enter an option: S

```

Figure 3: pfSense console showing detected interfaces (em0-em3) prior to assignment

Static IP addressing was then applied via the console: LAN (Segment A) at 192.168.15.1/24, OPT1 (Segment B) at 192.168.12.1/24, and OPT2 (Segment C) at 192.168.244.1/24, with WAN left on DHCP to obtain internet access through VMware's NAT.

```

You can now access the webConfigurator by opening the following URL in your web
browser:
        https://192.168.244.1/

Press <ENTER> to continue.
VMware Virtual Machine - Netgate Device ID: d8b4400c5bd652fc684f

*** Welcome to pfSense 2.8.1-RELEASE (amd64) on pfSense ***

WAN (wan)   -> em0 -> v4/DHCP4: 192.168.248.151/24
LAN (lan)   -> em1 -> v4: 192.168.15.1/24
OPT1 (opt1) -> em2 -> v4: 192.168.12.1/24
OPT2 (opt2) -> em3 -> v4: 192.168.244.1/24

0) Logout / Disconnect SSH          9) pfTop
1) Assign Interfaces                 10) Filter Logs
2) Set interface(s) IP address      11) Restart GUI
3) Reset admin account and password 12) PHP shell + pfSense tools
4) Reset to factory defaults        13) Update from console
5) Reboot system                    14) Enable Secure Shell (sshd)
6) Halt system                      15) Restore recent configuration
7) Ping host                        16) Restart PHP-FPM
8) Shell

Enter an option: S

```

Figure 4: pfSense console summary confirming all four interfaces addressed correctly

OPT1 and OPT2 were then enabled and renamed to SEGMENTB and SEGMENTC respectively through the pfSense web GUI, and DHCP was enabled on all three internal interfaces with a .100-.200 address pool on each subnet.

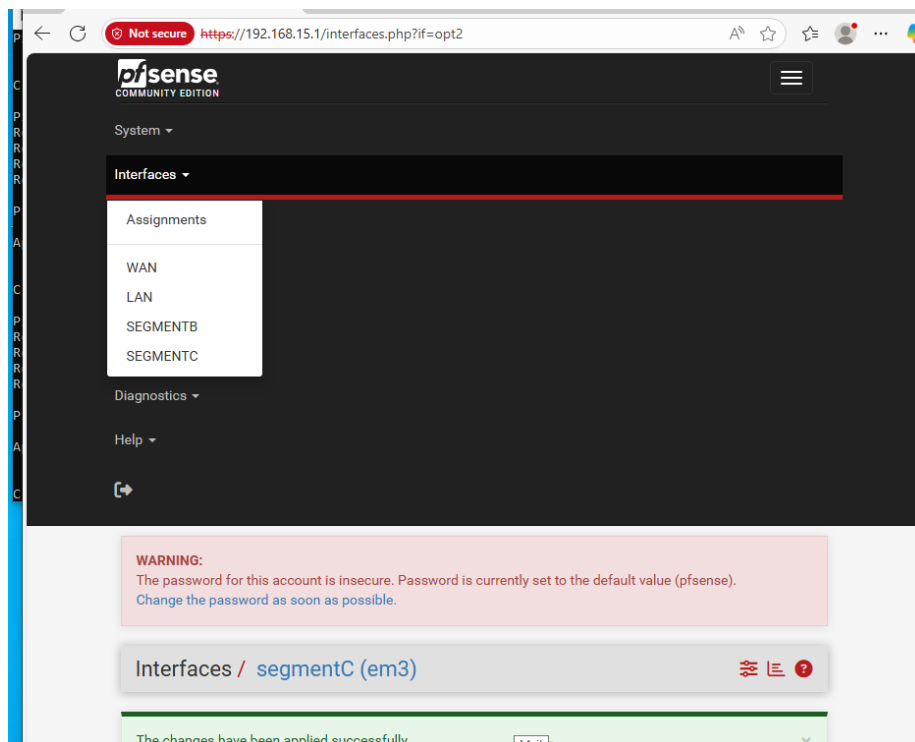


Figure 5: pfSense web GUI Interfaces menu confirming WAN, LAN, SEGMENTB, and SEGMENTC are all configured

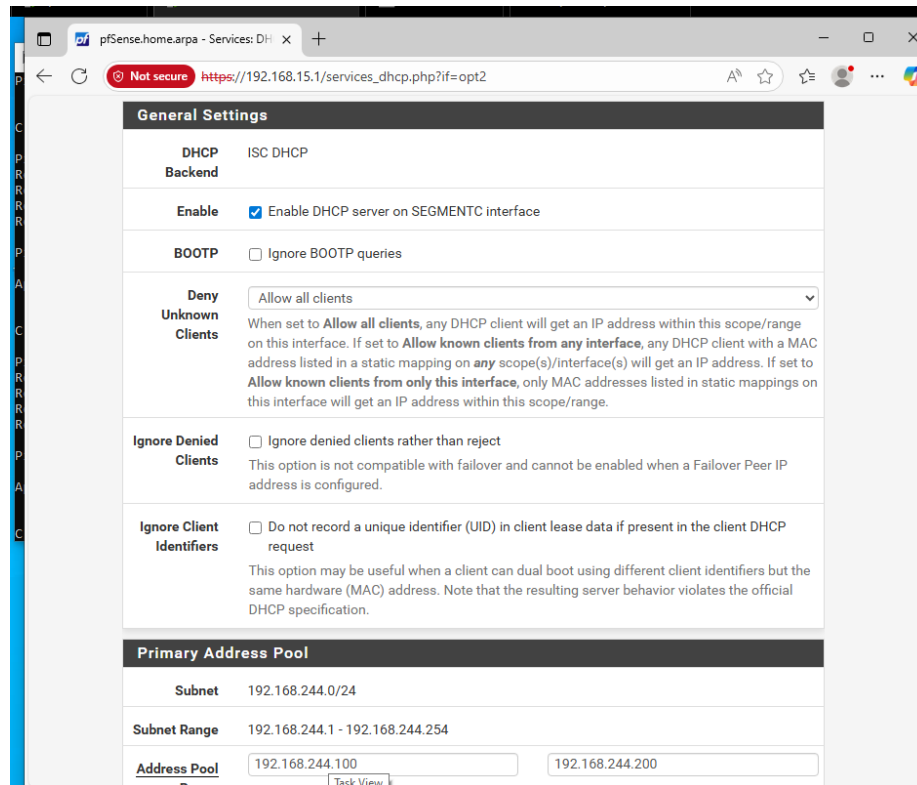


Figure 6: DHCP server configuration for Segment C (192.168.244.0/24), enabled with pool range 192.168.244.100-192.168.244.200

Outbound NAT was left on pfSense's default Automatic mode, and the default 'allow LAN to any' firewall rule (auto-created per interface) was confirmed present, giving each segment outbound connectivity without additional manual rules at this stage of the build.

4.2 Suricata Deployment

Suricata 8.0.6 was installed on an Ubuntu Server 22.04 VM via the OISF stable PPA, then configured with a per-segment HOME_NET and the sensor's actual NIC name. During the first sensor's setup, the default suricata.yaml still referenced the placeholder interface eth0, which does not exist under Ubuntu's predictable network naming; this was identified via a targeted grep and corrected to ens33, the sensor's real interface.

```

CPU: 7.615s
CGroup: /system.slice/suricata.service
└─2292 /usr/bin/suricata --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid --user suricata --group suricata

Aug 03 05:14:03 ubuntu-server systemd[1]: suricata.service: Failed with result 'exit-code'.
Aug 03 05:14:03 ubuntu-server systemd[1]: suricata.service: Consumed 26.529s CPU time over 26.644s wall clock time, 413.5M memory peak.
Aug 03 05:14:03 ubuntu-server systemd[1]: suricata.service: Scheduled restart job, restart counter is at 1.
Aug 03 05:14:03 ubuntu-server systemd[1]: Starting suricata.service - Suricata IDS/IPS/NSM/FW daemon...
Aug 03 05:14:03 ubuntu-server systemd[1]: Started suricata.service - Suricata IDS/IPS/NSM/FW daemon.
Aug 03 05:14:03 ubuntu-server suricata[2292]: i: suricata: This is Suricata version 6.0.6 RELEASE running in SYSTEM mode

tbotuo@ubuntu-server:~$ tail -f /var/log/suricata/eve.json
tail: Permission denied (os error 13) about ["/var/log/suricata"]
tbotuo@ubuntu-server:~$ sudo tail -f /var/log/suricata/eve.json
tbotuo@ubuntu-server:~$ sudo cat /var/log/suricata/eve.json | wc -l
0
tbotuo@ubuntu-server:~$ grep -A 3 "af-packet:" /etc/suricata/suricata.yaml
grep: /etc/suricata/suricata.yaml: Permission denied
tbotuo@ubuntu-server:~$ sudo grep -A 3 "af-packet:" /etc/suricata/suricata.yaml
af-packet:
- interface: eth0
  # Number of receive threads. "auto" uses the number of cores
  #threads: auto
tbotuo@ubuntu-server:~$

```

Figure 7: *suricata.yaml* confirmed to reference the non-existent *eth0* interface before correction to *ens33*

```

Expanded Security Maintenance for Applications is not enabled.

37 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

tbotuo@ubuntu-server:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:0c:29:9d:62:5c brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx00c299d625c
    inet 192.168.15.101/24 metric 100 brd 192.168.15.255 scope global dynamic ens33
        valid_lft 7121sec preferred_lft 7121sec
    inet6 fe80::20c:29ff:fe9d:625c/64 scope link proto kernel_l1
        valid_lft forever preferred_lft forever
tbotuo@ubuntu-server:~$

```

Figure 8: Segment A sensor network configuration (*ens33*, 192.168.15.101/24) used to set *HOME_NET* and the *af-packet* interface

After correcting the interface and updating *HOME_NET* to each segment's subnet, *suricata*-update was run to fetch the Emerging Threats Open ruleset (68,097 rules total, 52,158 enabled after default exclusions), and the service was enabled and verified as active.

```

tbotu@ubuntu-server:~$ sudo suricata-update
[sudo: authenticated] Password:
3/8/2026 -- 05:11:28 - <Info> -- Using data-directory /var/lib/suricata.
3/8/2026 -- 05:11:28 - <Info> -- Using Suricata configuration /etc/suricata/suricata.yaml
3/8/2026 -- 05:11:28 - <Info> -- Using /usr/share/suricata/rules for Suricata provided rules.
3/8/2026 -- 05:11:28 - <Info> -- Found Suricata version 8.0.6 at /usr/bin/suricata.
3/8/2026 -- 05:11:28 - <Info> -- Loading /etc/suricata/suricata.yaml
3/8/2026 -- 05:11:28 - <Info> -- Disabling rules for protocol postgres
3/8/2026 -- 05:11:28 - <Info> -- Disabling rules for protocol modbus
3/8/2026 -- 05:11:28 - <Info> -- Disabling rules for protocol dhcp
3/8/2026 -- 05:11:28 - <Info> -- Disabling rules for protocol snmp
3/8/2026 -- 05:11:28 - <Info> -- No sources configured, will use Emerging Threats Open
3/8/2026 -- 05:11:28 - <Info> -- Fetching https://rules.emergingthreats.net/open/suricata-8.0.6/emerging.rules.tar.gz.
100% - 6560583/5560583
3/8/2026 -- 05:11:33 - <Info> -- Done.
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/app-layer-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/decoder-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/dhcp-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/dns-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/files.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/http2-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/http-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/ipsec-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/kerberos-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/modbus-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/mqtt-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/nfs-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/ntp-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/quick-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/rfb-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/smb-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/smtp-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/ssh-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/stream-events.rules
3/8/2026 -- 05:11:34 - <Info> -- Loading distribution rule file /usr/share/suricata/rules/tls-events.rules
3/8/2026 -- 05:11:35 - <Info> -- Ignoring file 19f3e7870eada65a1b63663ccdbf/rules/emerging-deleted.rules
3/8/2026 -- 05:11:44 - <Info> -- Loaded 68097 rules.
3/8/2026 -- 05:11:45 - <Info> -- Disabled 16 rules.
3/8/2026 -- 05:11:45 - <Info> -- Enabled 0 rules.
3/8/2026 -- 05:11:45 - <Info> -- Modified 0 rules.
3/8/2026 -- 05:11:45 - <Info> -- Dropped 0 rules.
3/8/2026 -- 05:11:45 - <Info> -- Enabled 136 rules for flowbit dependencies.
3/8/2026 -- 05:11:45 - <Info> -- Backing up current rules.
3/8/2026 -- 05:11:45 - <Info> -- Writing rules to /var/lib/suricata/rules/suricata.rules: total: 68097; enabled: 52158; added: 6765
3/8/2026 -- 05:11:48 - <Info> -- Writing /var/lib/suricata/rules/classification.config
3/8/2026 -- 05:11:52 - <Info> -- Testing with suricata -T.
3/8/2026 -- 05:11:48 - <Info> -- Done.
3/8/2026 -- 05:11:52 - <Info> -- Testing with suricata -T.

```

Figure 9: suricata-update output confirming the Emerging Threats Open ruleset was fetched and loaded

```

3/8/2026 -- 05:11:45 - <Info> -- Backing up current rules.
3/8/2026 -- 05:11:45 - <Info> -- Writing rules to /var/lib/suricata/rules/suricata.rules: total: 68097; enabled: 52158; added: 6765; remove
3/8/2026 -- 05:11:48 - <Info> -- Writing /var/lib/suricata/rules/classification.config
3/8/2026 -- 05:11:52 - <Info> -- Testing with suricata -T.
3/8/2026 -- 05:12:47 - <Info> -- Done.
tbotu@ubuntu-server:~$ sudo systemctl enable --now suricata
tbotu@ubuntu-server:~$ sudo systemctl status suricata
* suricata.service - Suricata IDS/IPS/NSM/FW daemon
   Loaded: loaded (/usr/lib/systemd/system/suricata.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-08-03 05:14:03 UTC; 7s ago
     Invocation: 36465c1e85b84e95a2e93025c522430b
       Docs: man:suricata(8)
            man:suricatasc(8)
            https://suricata.io/documentation/
   Process: 2290 ExecStartPre=/bin/rm -f /run/suricata.pid (code=exited, status=0/SUCCESS)
  Main PID: 2292 (suricata-Main)
    Tasks: 1 (limit: 1827)
   Memory: 141.9M (peak: 141.9M)
      CPU: 7.615s
   CGroup: /system.slice/suricata.service
           └─2292 /usr/bin/suricata --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid --user suricata --group suricat

Aug 03 05:14:03 ubuntu-server systemd[1]: suricata.service: Failed with result 'exit-code'.
Aug 03 05:14:03 ubuntu-server systemd[1]: suricata.service: Consumed 26.529s CPU time over 26.644s wall clock time, 413.5M memory peak.
Aug 03 05:14:03 ubuntu-server systemd[1]: suricata.service: Scheduled restart job, restart counter is at 1.
Aug 03 05:14:03 ubuntu-server systemd[1]: Starting suricata.service - Suricata IDS/IPS/NSM/FW daemon...
Aug 03 05:14:03 ubuntu-server systemd[1]: Started suricata.service - Suricata IDS/IPS/NSM/FW daemon.
Aug 03 05:14:03 ubuntu-server suricata[2292]: i: suricata: This is Suricata version 8.0.6 RELEASE running in SYSTEM mode
tbotu@ubuntu-server:~$

```

Figure 10: systemctl status confirming the Suricata service active (running) on the Segment A sensor

Correct packet capture was validated by tailing the EVE JSON log while generating ICMP traffic from another host on the same segment; flow-type events referencing the correct source IP and inbound interface confirmed the sensor was genuinely inspecting live segment traffic rather than sitting idle.

```

tbotu@ubuntu-server:~$ sudo grep -o '"kernel_packet":{0-9}*' /var/log/suricata/eve.json | tail -5
tbotu@ubuntu-server:~$ sudo grep -o '"event_type": "flow"' /var/log/suricata/eve.json | tail -3
{"timestamp": "2026-08-03T05:49:53.814500000", "flow_id": "153399816486915", "in_iface": "ens33", "event_type": "flow", "src_ip": "192.168.15.50", "src_po
p": "192.168.248.2", "dest_port": "137", "ip_v": "4", "proto": "UDP", "app_proto": "failed", "flow": {"pkts_to_server": 0, "pkts_to_client": 0, "bytes_to_server": 1300, "
p": "start": "2026-08-03T05:49:17.553754000", "end": "2026-08-03T05:49:20.584639000", "age": 3, "state": "new", "reason": "timeout", "alerted": false}}
{"timestamp": "2026-08-03T05:51:53.260898000", "flow_id": "1532512369237463", "in_iface": "ens33", "event_type": "flow", "src_ip": "192.168.15.50", "src_po
p": "192.168.248.2", "dest_port": "137", "ip_v": "4", "proto": "UDP", "app_proto": "failed", "flow": {"pkts_to_server": 0, "pkts_to_client": 0, "bytes_to_server": 1300, "
p": "start": "2026-08-03T05:51:17.555517000", "end": "2026-08-03T05:51:20.584299000", "age": 3, "state": "new", "reason": "timeout", "alerted": false}}
{"timestamp": "2026-08-03T05:51:54.259479000", "flow_id": "820618885979886", "in_iface": "ens33", "event_type": "flow", "src_ip": "192.168.15.50", "src_po
p": "192.168.15.255", "dest_port": "138", "ip_v": "4", "proto": "UDP", "app_proto": "failed", "flow": {"pkts_to_server": 1, "pkts_to_client": 0, "bytes_to_server": 243, "
p": "start": "2026-08-03T05:51:14.977497000", "end": "2026-08-03T05:51:14.977497000", "age": 0, "state": "new", "reason": "timeout", "alerted": false}}
tbotu@ubuntu-server:~$

```

Figure 11: eve.json flow event confirming Suricata captured real traffic (src_ip 192.168.15.50) on interface ens33

The validated Segment A sensor VM was then cloned twice (full clones) to produce the Segment B and Segment C sensors, with each clone's network adapter reattached to the corresponding VMnet and HOME_NET updated to that segment's subnet.

```
tbotuo@ubuntu-server:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:0c:29:8f:65:34 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx00c298f6534
    inet 192.168.12.100/24 metric 1024 brd 192.168.12.255 scope global dynamic ens33
        valid_lft 7004sec preferred_lft 7004sec
    inet6 fe80::20c:29ff:fe8f:6534/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
tbotuo@ubuntu-server:~$
```

Figure 12: Segment B sensor after cloning, confirmed on 192.168.12.100/24 via DHCP from pfSense's SEGMENTB interface

```
SHELLCODE_PORTS: "180"
ORACLE_PORTS: 1521
SSH_PORTS: 22
DNP3_PORTS: 20000

tbotuo@ubuntu-server:~$ sudo systemctl restart suricata
tbotuo@ubuntu-server:~$ sudo systemctl status suricata
* suricata.service - Suricata IDS/IPS/NSM/FW daemon
   Loaded: loaded (/usr/lib/systemd/system/suricata.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-08-03 06:27:44 UTC; 21s ago
   Invocation: 4944a7835f6149779158e5e0963dcb17
     Docs: man:suricata(8)
           man:suricatasc(8)
           https://suricata.io/documentation/
   Process: 2149 ExecStartPre=/bin/rm -f /run/suricata.pid (code=exited, status=0/SUCCESS)
   Main PID: 2151 (Suricata-Main)
     Tasks: 1 (limit: 1827)
    Memory: 270.1M (peak: 270.1M)
       CPU: 21.160s
   CGroup: /system.slice/suricata.service
           └─2151 /usr/bin/suricata --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid

Aug 03 06:27:44 ubuntu-server systemd[1]: Starting suricata.service - Suricata IDS/IPS/NSM/FW daemon...
Aug 03 06:27:44 ubuntu-server systemd[1]: Started suricata.service - Suricata IDS/IPS/NSM/FW daemon.
tbotuo@ubuntu-server:~$
```

Figure 13: Segment B Suricata service active (running) after HOME_NET was updated to 192.168.12.0/24

The same clone-and-reconfigure process was repeated for the Segment C sensor (HOME_NET set to 192.168.244.0/24), which was confirmed running using the same verification steps.

4.3 Connectivity Troubleshooting (Evidence)

The deviations noted in Section 3.5 are evidenced here. Diagnostics included checking pfSense's ARP table (confirming Layer 2 resolution was intact even while ICMP failed), and reviewing DNS resolver configuration on the affected sensor.

Interface	IP Address	MAC Address	Hostname	Status	Link Type	Actions
WAN	192.168.248.151	00:0c:29:1f:72:e0		Permanent	ethernet	  
WAN	192.168.248.254	00:50:56:ed:34:6f		Expires in 435 seconds	ethernet	  
WAN	192.168.248.2	00:50:56:f7:fc:c7		Expires in 1038 seconds	ethernet	  
LAN	192.168.15.50	00:0c:29:2d:c5:77		Expires in 1129 seconds	ethernet	  
SEGMENTB	192.168.12.1	00:0c:29:1f:72:f4		Permanent	ethernet	  
SEGMENTC	192.168.244.1	00:0c:29:1f:72:fe		Permanent	ethernet	  
LAN	192.168.15.1	00:0c:29:1f:72:ea	pfSense.home.arpa	Permanent	ethernet	  
LAN	192.168.15.101	00:0c:29:9d:62:5c	ubuntu-server	Expires in 1153 seconds	ethernet	  



 Local IPv6 peers use NDP instead of ARP.

Figure 14: pfSense ARP table showing correct MAC resolution for the Ubuntu sensor despite ICMP failing, pointing to an offload-related issue rather than an addressing or Layer-2 fault

pfSense
COMMUNITY EDITION

WARNING:
The password for this account is insecure. Password is currently set to the default value (pfsense).
[Change the password as soon as possible.](#)

Firewall / NAT / Outbound

Port Forward 1:1 Outbound NAT

Outbound NAT Mode

Mode

☒ Automatic outbound NAT rule generation. (IPsec passthrough included)
☐ Hybrid Outbound NAT rule generation. (Automatic Outbound NAT + rules below)
☐ Manual Outbound NAT rule generation. (AON - Advanced Outbound NAT)
☐ Disable Outbound NAT rule generation. (No Outbound NAT rules)



Mappings

☐	Interface	Source	Destination	NAT Address	NAT Port	Static Port	Description	Actions
	Source	Port	Destination	Port				

Figure 15: Outbound NAT confirmed on Automatic mode, ruling out a NAT misconfiguration as the cause

```

    valid_lft forever preferred_lft forever
inet6 fe80::20c:29ff:fe9d:625c/64 scope link proto kernel_ll
    valid_lft forever preferred_lft forever
tbotuo@ubuntu-server:~$ ping 192.168.15.1
PING 192.168.15.1 (192.168.15.1) 56(84) bytes of data:
64 bytes from 192.168.15.1: icmp_seq=1 ttl=64 time=0.833 ms
64 bytes from 192.168.15.1: icmp_seq=2 ttl=64 time=0.961 ms
64 bytes from 192.168.15.1: icmp_seq=3 ttl=64 time=7.84 ms
64 bytes from 192.168.15.1: icmp_seq=4 ttl=64 time=2.48 ms
64 bytes from 192.168.15.1: icmp_seq=5 ttl=64 time=1.07 ms
64 bytes from 192.168.15.1: icmp_seq=6 ttl=64 time=0.833 ms
^C
--- 192.168.15.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5009ms
rtt min/avg/max/mdev = 0.833/2.835/7.835/2.525 ms
tbotuo@ubuntu-server:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data:
64 bytes from 8.8.8.8: icmp_seq=1 ttl=127 time=164 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=127 time=164 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=127 time=169 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=127 time=159 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=127 time=167 ms
^C
--- 8.8.8.8 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4004ms
rtt min/avg/max/mdev = 158.928/164.679/169.264/3.405 ms
tbotuo@ubuntu-server:~$ ping google.com
PING google.com (216.58.223.206) 56(84) bytes of data:
64 bytes from los02s03-in-f14.1e100.net (216.58.223.206): icmp_seq=1 ttl=127 time=150 ms
64 bytes from los02s03-in-f14.1e100.net (216.58.223.206): icmp_seq=2 ttl=127 time=163 ms
64 bytes from los02s03-in-f14.1e100.net (216.58.223.206): icmp_seq=3 ttl=127 time=187 ms
64 bytes from los02s03-in-f14.1e100.net (216.58.223.206): icmp_seq=4 ttl=127 time=188 ms
64 bytes from los02s03-in-f14.1e100.net (216.58.223.206): icmp_seq=5 ttl=127 time=152 ms
64 bytes from los02s03-in-f14.1e100.net (216.58.223.206): icmp_seq=6 ttl=127 time=161 ms
^C
--- google.com ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 9166ms
rtt min/avg/max/mdev = 149.604/166.690/190.267/15.140 ms

```

Figure 16: Full connectivity (gateway, public IP, and DNS) confirmed working after disabling hardware offload and rebooting both the pfSense and Ubuntu VMs

8. References

- [1] Open Information Security Foundation, "Suricata User Guide," Suricata Documentation. [Online]. Available: <https://docs.suricata.io/>
- [2] Wazuh, Inc., "Wazuh Documentation," Wazuh. [Online]. Available: <https://documentation.wazuh.com/>
- [3] MITRE Corporation, "MITRE ATT&CK," MITRE ATT&CK Framework. [Online]. Available: <https://attack.mitre.org/>
- [4] Center for Internet Security, "CIS Benchmarks," CIS. [Online]. Available: <https://www.cisecurity.org/cis-benchmarks>
- [5] Netgate, "pfSense Documentation," pfSense Docs. [Online]. Available: <https://docs.netgate.com/pfsense/en/latest/>